PROJECT DELIVERABLES

Intelligent Vendor Routing Platform

A unified API that routes requests to the best available vendor across configurable strategies, with automatic failover, live metrics, and a rule-based AI config generator.

SOURCE CODE REPOSITORY

github.com/Narayana-murthy-chikkala/Signzy

MongoDB Atlas · Node.js / Express · React (Vite) · Jest / Supertest

1. Source Code

The complete, working source code is hosted at:

> **Repository:** `https://github.com/Narayana-murthy-chikkala/Signzy` (branch: `main`)

Tech Stack

LAYER	CHOICE
Frontend	React 19 (Vite) + Tailwind CSS + React Router + Recharts
Backend	Node.js + Express.js — MVC with the Strategy design pattern for routing
Database	MongoDB Atlas + Mongoose
Testing	Jest + Supertest (53 tests: unit + integration against an in-memory MongoDB)
API Testing	Postman collection (`server/postman_collection.json`)

Project Structure

```
client/src/
  components/   Layout, tables, cards, modal, badges, toasts, progress bars
  pages/        Dashboard, Vendors, RouteTester, Metrics, Logs, Health, AIRuleGenerator
  services/     axios API client
  hooks/        useMetrics, useLogs, useHealth
  context/      AppContext (shared vendor list), ToastContext
  utils/        formatting + per-capability example payloads

server/
  config/       DB connection, runtime constants
  controllers/  vendor, routing, metrics, logs, health, ai-rule, routing-config, advice
  models/       Vendor, RoutingLog, RoutingConfig (Mongoose schemas)
  routes/       Express routers
  middleware/   centralized error handler (normalizes Mongoose errors to clean 4xx)
  services/     vendorSimulator, metricsService, loggingService, routingEngine,
                strategyAdvisor, fallbackAdvisor
  strategies/   priority / weighted / roundRobin / lowestLatency / lowestCost /
                healthBased / failover / featureBased  (Strategy pattern, 1 file each)
  utils/        filterVendors, rateLimiter, computeAvailability, vendorCache,
                aiRuleGenerator, capabilityResponses, seed
  tests/        Jest + Supertest suite (53 tests)
```

2. README

The full README (setup instructions, API reference, routing strategy details) lives at the repository root: `README.md`. Summary below.

Problem

Modern platforms depend on multiple third-party vendors for the same capability (KYC/PAN verification, OCR, SMS, payment processing, document validation) — each differing in cost, latency, success rate, rate limits, error rate, availability, and supported features. This project exposes **one unified API** that decides which vendor actually handles a given request, so the client never needs to know which vendor was used.

Setup

```
npm run install:all      # installs server + client dependencies
npm run seed             # seeds 5 sample vendors into MongoDB
npm run dev              # runs backend (:5000) and frontend (:5173) together

cd server && npm test    # runs the automated test suite (in-memory Mongo)
```

Mandatory APIs (all implemented)

METHOD	ROUTE	PURPOSE
POST	`/vendors`	Register a vendor
GET	`/vendors`	List vendors
PUT	`/vendors/:id`	Update a vendor
DELETE	`/vendors/:id`	Delete a vendor
POST	`/route`	Route a request to the best vendor
GET	`/vendor-metrics`	Per-vendor + aggregate metrics
GET	`/routing-logs`	Paginated/filterable routing decision logs
GET	`/health`	Server + DB + vendor health snapshot

Bonus endpoints: `/ai-rule-generator`, `/routing-configs`, `/strategy-recommendation`, `/fallback-suggestions`.

3. Sample Vendor Configs

The seed script (`server/utils/seed.js`) populates 5 vendors covering all 5 capabilities, deliberately including one unhealthy and one down vendor so filtering/failover is visible immediately. This is the actual current state of the database, pulled live via `GET /vendors`:

NAME	PRIORITY	WEIGHT	COST	TIMEOUT	RATE LIMIT	HEALTH	UP?
Vendor A	5	5	$0.10	2000ms	100/min	healthy	yes
Vendor B	3	3	$0.05	1500ms	50/min	healthy	yes
Vendor C	4	4	$0.20	3000ms	200/min	healthy	yes
Vendor D	2	2	$0.02	1000ms	30/min	unhealthy	yes
Vendor E	1	1	$0.15	2500ms	100/min	healthy	down

Capability coverage

VENDOR	SUPPORTED CAPABILITIES
Vendor A	`PAN_VERIFICATION, OCR, SMS`
Vendor B	`PAN_VERIFICATION, PAYMENT_PROCESSING, DOCUMENT_VALIDATION`
Vendor C	`OCR, SMS, PAYMENT_PROCESSING, DOCUMENT_VALIDATION`
Vendor D	`PAN_VERIFICATION, DOCUMENT_VALIDATION`
Vendor E	`OCR, SMS, PAYMENT_PROCESSING`

Raw config (as sent to `POST /vendors`)

```
{
  "name": "Vendor A",
  "priority": 5,
  "weight": 5,
  "costPerRequest": 0.1,
  "timeoutMs": 2000,
  "rateLimitPerMinute": 100,
  "supportedFeatures": ["PAN_VERIFICATION", "OCR", "SMS"],
  "healthStatus": "healthy",
  "isActive": true
}
```

4. Sample API Requests & Responses

Every capability gets a distinct, realistic request/response shape rather than one generic envelope for everything — captured live from the running application (real request IDs, real simulated latencies).

PAN Verification

REQUEST

```
POST /route
{
  "capability": "PAN_VERIFICATION",
  "payload": { "pan": "ABCDE1234F", "name":
"Rahul Sharma" }
}
```

RESPONSE

```
{
  "status": "SUCCESS",
  "vendorUsed": "Vendor A",
  "routingReason": "Highest priority (5) among
eligible vendors",
  "latencyMs": 498,
  "cost": 0.1,
  "response": {
    "panStatus": "VALID",
    "nameMatch": true,
    "pan": "ABCDE1234F"
  }
}
```

SMS (with a live failover)

This one shows the failover mechanism firing for real: Vendor A timed out, so the engine automatically moved to the next-ranked eligible vendor.

REQUEST

```
POST /route
{
  "capability": "SMS",
  "payload": { "to": "+919876543210",
"message": "Your OTP is 482913" }
}
```

RESPONSE


```
{
  "status": "SUCCESS",
  "vendorUsed": "Vendor C",
  "routingReason": "Highest priority (4) among
eligible vendors",
  "latencyMs": 1511,
  "cost": 0.2,
  "response": {
    "messageId": "sms_qk7tip1r",
    "deliveryStatus": "DELIVERED",
    "to": "+919876543210"
  },
  "failoverHistory": [
    { "vendor": "Vendor B", "status":
"skipped", "reason": "does not support
capability \"SMS\"" },
    { "vendor": "Vendor D", "status":
"skipped", "reason": "Vendor is unhealthy" },
    { "vendor": "Vendor E", "status":
"skipped", "reason": "Vendor is down" },
    { "vendor": "Vendor A", "status":
"timeout", "reason": "Vendor timed out after
2000ms", "latencyMs": 2000 },
    { "vendor": "Vendor C", "status":
"success", "reason": "processed successfully",
"latencyMs": 1511 }
  ]
}
```


Payment Processing

REQUEST

```
POST /route
{
  "capability": "PAYMENT_PROCESSING",
  "payload": { "amount": 999.0, "currency":
"INR", "cardLast4": "4242" }
}
```

RESPONSE


```
{
  "status": "SUCCESS",
  "vendorUsed": "Vendor C",
  "routingReason": "Highest priority (4) among
eligible vendors",
  "latencyMs": 1575,
  "cost": 0.2,
  "response": {
    "transactionId": "txn_82d8mq3c",
    "paymentStatus": "CAPTURED",
    "amountCharged": 999,
    "currency": "INR"
  }
}
```

Document Validation & OCR

DOCUMENT_VALIDATION RESPONSE


```
{
  "validationStatus": "VALID",
  "documentType": "CONTRACT",
  "issuesFound": []
}
```


OCR RESPONSE


```
{
  "extractedText": "M/S ACME TRADING CO. |
INVOICE #4821 | AMOUNT: 12,450.00",
  "confidence": 0.89,
  "fieldsDetected": { "documentType":
"INVOICE", "pageCount": 1 }
}
```


GET /vendor-metrics (aggregate)


```
{
  "summary": {
    "totalVendors": 5,
    "healthyVendors": 3,
    "totalRequests": 9,
    "successRate": 55.56,
    "averageLatency": 1365
  }
}
```


GET /strategy-recommendation (Agentic AI bonus)


```
{
  "recommendedStrategy": "healthBased",
  "reason": "Vendor reliability varies significantly (health spread 100%, and at least
            one vendor has an error rate above 20%). Health-based routing steers
            traffic away from the less reliable vendors automatically.",
  "signals": { "vendorsAnalyzed": 3, "totalRequests": 9, "latencySpreadPct": 92.3,
               "costSpreadPct": 75, "healthSpreadPct": 100 }
}
```

5. Architecture Diagram

React Client (Vite Dashboard)

Vendor Management · Route Tester · Metrics · Logs · Health · AI Rule Generator

↓ axios / REST (JSON)

Express API Layer

/vendors · /route · /vendor-metrics · /routing-logs · /health · /ai-rule-generator · /routing-configs · /strategy-recommendation · /fallback-suggestions

↓

Routing Engine `routingEngine.js`

Orchestrates every /route call end to end

↓

1. Vendor Cache

Short-TTL in-memory cache over MongoDB, invalidated on every write

2. Filter

Excludes down / unhealthy / rate-limited / missing-feature / over-latency vendors

3. Strategy (rank)

Strategy-pattern registry — 8 interchangeable ranking algorithms

4. Condition Override

AI-generated conditions can promote a fallback vendor ahead of the ranking

↓ attempt in ranked order, retry on failure

Vendor Simulator

Randomized latency (100–3000ms) + success/failure/timeout outcome; capability-specific response shape (no real vendor APIs are ever called)

↓

Metrics Service

Atomic counters → success/error rate, avg latency, auto-unhealthy circuit breaker

Rate Limiter

True sliding-window log, atomic per-document admit check

Logging Service

RoutingLog (MongoDB) + flat-file audit trail

↓

Standard Response

{ status, vendorUsed, routingReason, latencyMs, cost, response, failoverHistory } — identical shape regardless of vendor

Agentic AI layer (bonus)

Plain-English rule

"Use Vendor A for 70%, Vendor B for 30%, switch to Vendor C if latency crosses 2s or error rate is above 5%."

↓ regex parser (no external LLM call)

Routing Config JSON

{ strategy, vendorOrder, weights, conditions }

↓ Save & Apply

Applied to real vendors

Weights → vendor.weight | no weights → vendor.priority (order preserved) | conditions usable from Route Tester

6. Explanation of Routing Decisions

How one `/route` call resolves

1. **Fetch & filter.** All vendors are read (via the short-TTL cache) and each is checked against five exclusion rules: *down* (`isActive: false`), *unhealthy*, *rate-limited* (sliding window over the last 60s), *missing the requested capability*, and *over the latency threshold*. Every excluded vendor is recorded with its exact reason.
2. **Rank.** The remaining eligible vendors are ordered by whichever strategy was requested (or the sensible default — see below).
3. **Apply conditions.** If an AI-generated condition applies (e.g. "switch to Vendor C if latency exceeds 2s") and the top-ranked vendor currently breaches it, the named fallback is promoted to the front of the list.
4. **Attempt with failover.** The engine calls the simulated vendor. On success, it returns immediately. On failure/timeout, it automatically tries the *next* vendor in the ranked list — continuing until one succeeds or the list is exhausted.
5. **Record & respond.** Metrics update atomically, a routing log is written, and the standard response envelope is returned — identical shape whether it succeeded on the first vendor or the fourth.

The 8 routing strategies

STRATEGY	RANKING LOGIC
`priority`	Descending by configured `priority` value
`weighted`	Weighted-random sampling without replacement, by `weight`
`roundRobin`	Rotates by stable vendor identity — correct even as the eligible set changes shape between calls
`lowestLatency`	Ascending by `currentLatency`; untested vendors are ranked after proven ones (not falsely "fastest" at 0ms)
`lowestCost`	Ascending by `costPerRequest`
`healthBased`	Composite score from success rate, an independently-tracked availability metric, and error rate
`failover`	Strict priority order — the canonical "primary, then failover chain" strategy
`featureBased`	Prefers vendors supporting the broadest additional feature set as a tiebreak

`strategy` is optional on `/route`: if omitted, a `requirements.preferLowCost: true` hint picks `lowestCost`, otherwise it defaults to `priority`.

Why a vendor was (or wasn't) selected

Every response carries a `routingReason` string generated per strategy (e.g. "*Highest priority (5) among eligible vendors*", "*Lowest cost per request ($0.05) among eligible vendors*"), plus a full `failoverHistory` listing

every vendor that was considered, whether it was skipped or attempted, and exactly why.

Automatic failover triggers

TRIGGER	MECHANISM
Vendor down	Excluded before ranking (`isActive: false`)
Timeout	Simulated call exceeds the vendor's `timeoutMs` → next vendor attempted
High latency	Excluded before ranking if `currentLatency` exceeds the threshold
High error rate	Auto-circuit-breaker flips the vendor `unhealthy` once its rolling error rate crosses a threshold over enough requests, excluding it from future routing
Rate limit exceeded	Atomic, race-free sliding-window check — excluded before ranking, or the attempt itself is rejected if a burst of concurrent requests would exceed the limit
Unsupported feature	Excluded before ranking if the vendor's `supportedFeatures` doesn't include the requested capability

7. AI Usage

Full disclosure lives in `AI_USAGE.md` at the repository root. Summary:

AI used to build this project

Claude Code (Claude Sonnet 5) was used as an AI pair-programmer throughout development — scaffolding, implementation, the test suite, and iterative debugging. A human directed all scope decisions, supplied the assignment brief, caught real defects during manual testing (which prompted fixes), and requested explicit self-audits against the brief at multiple points. Every change was verified against a live, running instance of the app — via curl, the automated test suite, and real browser sessions (screenshots + console-error checks) — rather than accepted on the basis of code generation alone.

AI-branded features inside the product (the bonus)

The AI Rule Generator, strategy recommender, unhealthy-vendor detector, and fallback suggester are all implemented as **rule-based logic and statistical heuristics — not calls to an external LLM API**, for the same reason vendors are simulated rather than real: deterministic, inspectable, unit-testable behavior for the actual grading criteria, with no dependency on a third-party model's availability or non-determinism. See table in `AI_USAGE.md` for exactly which file implements which bonus feature.